

=> Erklärung von Py Skripte:

Linear Regression

Wir laden NumPy für die Matrizenrechnung

import numpy as np

Definition unserer Funktion für die manuelle Regression

def linear_regression_simple(X, y):

Berechnung der Mittelwerte von X und y
 # $\bar{x} = \frac{1}{n} \sum x_i$ und $\bar{y} = \frac{1}{n} \sum y_i$

x_mean = np.mean(X)
 y_mean = np.mean(y)

Der Zähler: Bestimmung der Kovarianz

$\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$

numerator = np.sum((X - x_mean) * (y - y_mean))

Der Nenner: Die Varianz von X

$\sum_{i=1}^n (x_i - \bar{x})^2$

denominator = np.sum((X - x_mean) ** 2)

Berechnung der Steigung (m) $y = mx + b$

slope = numerator / denominator

Berechnung des Achsenabschnitts (b)

$b = \bar{y} - m\bar{x}$

intercept = y_mean - slope * x_mean

Rückgabe der Parameter für unsere Gerade!

return slope, intercept

Logistic Regression

Die Sigmoid-Funktion: Sie quetscht jeden wert in den Bereich zwischen 0 und 1.

$\sigma(z) = \frac{1}{1 + e^{-z}}$ $\Rightarrow$ 

"Tab"

* Einrücken

def sigmoid(z):

return 1/(1 + np.exp(-z))

Unserer logistische Regression Funktion

def logistic_regression_simple(X, y, learning_rate=0.1, epochs=1000):

Wir ermitteln die Anzahl der Datensätze (m), und Merkmale (n).

m, n = X.shape

Wir fügen eine künstliche Spalte hinzu, damit der Bias wie ein normales Gewicht behandelt werden kann.

Beispiel: Dimensionen müssen passen!

$\theta = [b, w_1, w_2]$ $X = [1, x_1, x_2]$

$X \cdot \text{dot}(\theta)$: $[1, x_1, x_2] \cdot [b, w_1, w_2] = (1 \cdot b) + (x_1 \cdot w_1) + (x_2 \cdot w_2)$

X = np.c_[np.ones(m), X]
theta = np.zeros(n+1)

Die Trainings-Schleife: Wir wiederholen den Lernprozess mehrmals.

for _ in range(epochs):

Prediction $\Rightarrow$ Wir berechnen die aktuelle Wahrscheinlichkeiten

$\hat{y} = \sigma(X\theta)$.

predictions = sigmoid(X.dot(theta))

Fehleranalyse $\Rightarrow$ Wie weit liegen wir neben der Wahrheit?

errors = predictions - y

Gradient $\Rightarrow$ Wir berechnen die Richtung der steilste Korrektur

$\nabla = \begin{pmatrix} \frac{\partial F}{\partial x} \\ \frac{\partial F}{\partial y} \\ \frac{\partial F}{\partial z} \end{pmatrix}$

Beispiel von eine Gradient $F(x, y, z)$!

gradients = (1/m) * X.T.dot(errors)

Transpose ist (T) => $v = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \Rightarrow v.T = [x \ y \ z]$

#

Gewichtsupdate => Wir machen einen kleinen Schritt in die
richtige Richtung

$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot \nabla J$

theta -= learning_rate * gradients

+= , -= sind immer so: $x = x + b$
oder
$x += b$

<# Ausrücken

Rückgabe unsere θ_{new}

return theta

Perceptron

Die ReLU-Aktivierungsfunktion: "Türsteher" des Neurons.

$f(x) = \max(0, x)$

def relu(x):

return max(0, x)

Forward-Pass Funktion

def perceptron_forward_pass(x, weights, bias):

Die gewichtete Summe: Jeder Merkmal wird bewertet und der Bias kommt hinzu.

$z = \sum(x_i \cdot w_i) + b$

linear_output = np.dot(x, weights) + bias

Die Entscheidung: Das Ergebnis wird durch ReLU aktiviert.

prediction = relu(linear_output)

Rückgabe des Ergebnisses

return prediction

$$\text{Dot Product} \begin{pmatrix} c \\ d \end{pmatrix} \begin{pmatrix} a & b \end{pmatrix} = ac + bd$$

Initialization und Runtime $\Rightarrow$ Das Modell startet ohne jegliches Wissen

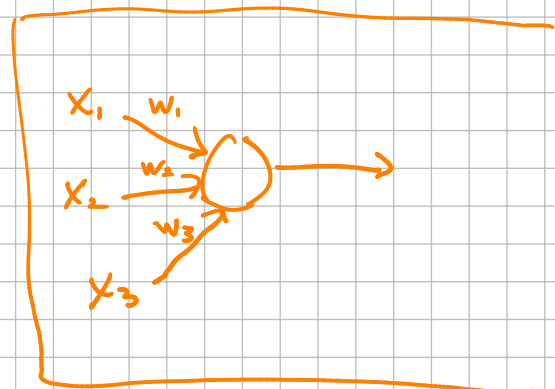
x = np.array([1, 0, 1, 0, 1])
y = 1

Daten erstellen
Label erstellen

weights = np.zeros(len(x))
bias = 0

Erste Testlauf $\Rightarrow$ Berechnung des Outputs
mit den Startwerten

output = perceptron_forward_pass



Lernen Mechanismus: Iterativ

```
def perceptron_learning(x, y, learning_rate=0.1, epochs=10):
```

```
    weights = np.zeros(len(x))  
    bias = 0
```

Der Training-Zyklus $\Rightarrow$ Wir wiederholen den Lernschritt bis das
Modell stabil ist.

```
    for epoch in range(epochs):
```

```
        linear_output = np.dot(x, weights) + bias  
        prediction = relu(linear_output)
```

Die Abweichung $\Rightarrow$ Was ist die Unterschied zwischen Wunsch und
Wirklichkeit?

```
#  $E = y - \hat{y}$ 
```

```
    error = y - prediction
```

Gewichts Anpassung $\Rightarrow$ die Gewichte werden in Richtung des
Fehlers verschoben

```
#  $\Delta w = \eta \cdot (y - \hat{y}) \cdot x$ 
```

```
    weights += learning_rate * error * x
```

Bias-Korrektur $\Rightarrow$ die Aktivierungsschwelle wird angepasst.

```
#  $\Delta b = \eta \cdot (y - \hat{y})$ 
```

```
    bias += learning_rate * error
```

Rückgabe unsere Parameter (Gewichte, Bias)

```
    return weights, bias
```

Execution und Output

```
final_weights, final_bias = perceptron_learning(x, y)
```

Fortschrittskontrolle $\Rightarrow$ Wir beobachten wie der Fehler mit jedem
Schritt kleiner wird

```
print(f"Epoch {epoch + 1}: ... Error = {error}")
```

Single linear Neuron:

Mathematische Beispiel

Given

$$x = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$$

$$w = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$b = 0$$

$$y = 3$$

⇒ Compute the forward pass:

$$\hat{y} = w^T x + b$$

Loss:

$$L = \frac{1}{2} (\hat{y} - y)^2$$

$$\Rightarrow \hat{y} = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} = 2 + 0 = \textcircled{2} \checkmark$$

$$\Rightarrow L = \frac{1}{2} (2 - 3)^2 = \textcircled{\frac{1}{2}} \checkmark$$

Berechnung von Gradienten

$$\frac{\partial L}{\partial w} \quad \frac{\partial L}{\partial b}$$

$$\Rightarrow L = \frac{1}{2} (\hat{y} - y)^2$$

$$L = \frac{1}{2} [(w^T x + b) - y]^2$$

$$\frac{\partial L}{\partial w} = 2 \left(\frac{1}{2} \right) (w^T x + b - y) x$$

$$= (2 - 3) x = -x$$

$$\frac{\partial L}{\partial w} = \begin{pmatrix} -2 \\ 1 \end{pmatrix}$$

$$\frac{\partial L}{\partial b} = 2 \left(\frac{1}{2} \right) (w^T x + b - y) (1)$$

$$\frac{\partial L}{\partial b} = (2 - 3) (1) = -1$$

Using Learning rate $\eta = 0.1$,

compute parameter w_{new} , und b_{new}

$$\nabla_{\theta} L = \frac{\partial L}{\partial w}$$

$$\theta = w$$

$\Rightarrow$ Wir verwenden $\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L$

$$\Rightarrow w_{\text{new}} = w_{\text{old}} - (0.1) \begin{pmatrix} -2 \\ 1 \end{pmatrix}$$

$$w_{\text{new}} = \begin{pmatrix} 1.2 \\ -0.1 \end{pmatrix}$$

$$\Rightarrow b_{\text{new}} = b_{\text{old}} - 0.1 \frac{\partial L}{\partial b} = 0 - (0.1)(-1) = 0.1$$